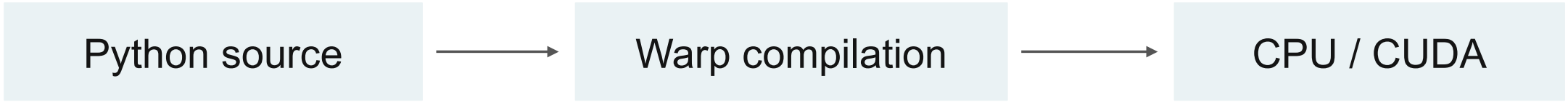
Data-driven Computer Animation

Tutorial 5 – Introduction to NVIDIA Warp

Prof. Taku Komura
TAs: Zhouyingcheng Liao (zycliao@cs.hku.hk)

What is NVIDIA Warp?

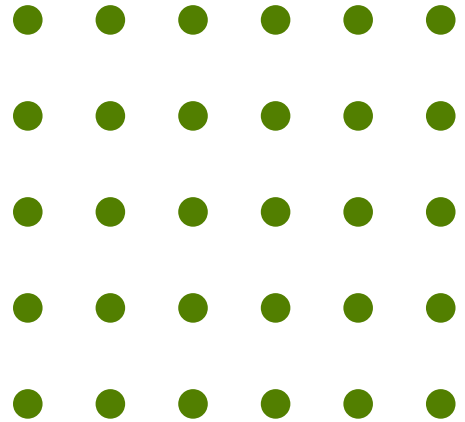
- A Python framework for simulation and spatial computing.
- Write typed kernels; compile them for CPU or NVIDIA GPU execution.
- Use arrays, vectors and matrices to describe numerical work.



Today: data → one thread per element → reusable kernels.

Why Warp for computer animation?

- Many particles or vertices repeat the same calculation.
- A kernel describes the work for one element.
- Warp also supports gradients for differentiable computations.



many elements, same rule

Our running example: falling particles bouncing on a flat ground.

Python vs Warp

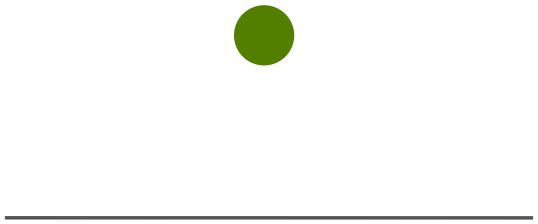
Python scope	Kernel scope
Create data and arrays Select a device Launch kernels Plot results	→ Read the thread index Load element data Do typed numerical work Write results

Decorators mark compiled code; wp.launch() schedules its execution.

How does a falling ball move?

Now → Advance by 0.1 s → Next

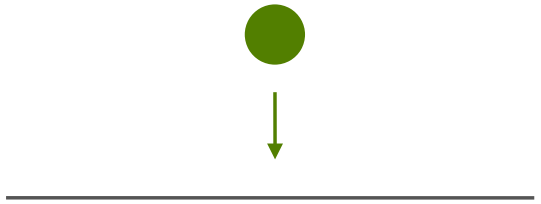
height = 1.000 m
velocity = 0.000 m/s



Gravity changes velocity.
Velocity changes height.
Then check the ground.

gravity = −9.81 m/s²

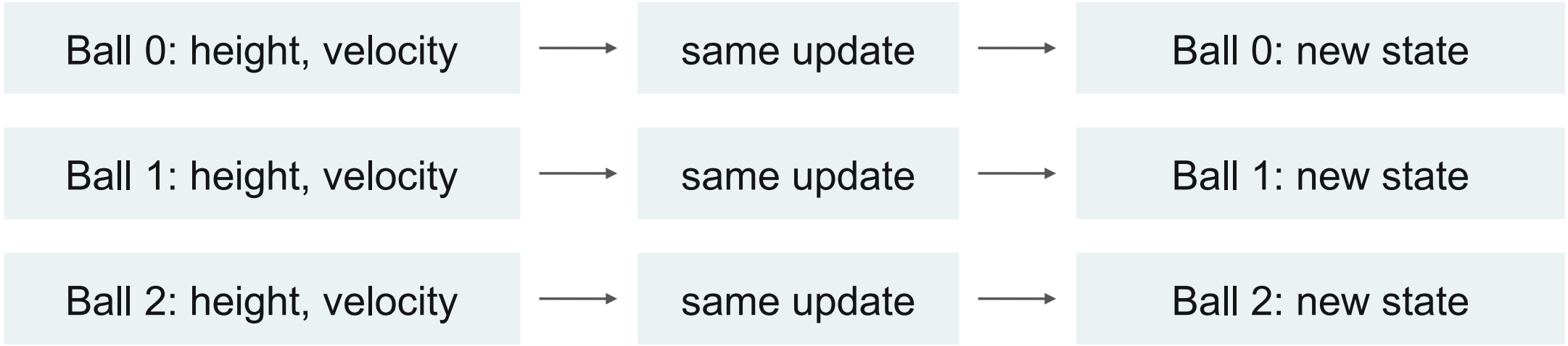
height ≈ 0.902 m
velocity = −0.981 m/s



Repeat this small update to generate motion. The next state becomes the current state.

Many balls repeat the same update

Here, balls touch the ground but do not collide with each other.



Each update reads one ball and writes that same ball.

These updates can run in parallel. We will implement the rule once.

Installation

Official installation guide

https://nvidia.github.io/warp/stable/user_guide/installation.html

wp.init() and device selection

- Initialization reports available devices and the kernel cache.

```
import warp as wp

wp.init()
print(wp.get_devices())
device = "cuda:0" if wp.is_cuda_available() else "cpu"

x = wp.zeros(1024, dtype=wp.vec3, device=device)
```

Use the same device for arrays and the kernel that accesses them.

Selecting a GPU

- Choose visibility before starting Python.
- Within Python, select a visible device with "cuda:0", "cuda:1", ...

```
# Shell: expose physical GPU 2 to this process
export CUDA_VISIBLE_DEVICES=2
python

# Python: logical device 0 is physical GPU 2 in this process
device = "cuda:0"
```

Always check the initialization log to confirm the selected device.

Your first kernel

- Compute one output element for each logical thread.

```
import warp as wp

@wp.kernel
def add(a: wp.array(dtype=float),
        b: wp.array(dtype=float),
        out: wp.array(dtype=float)):
    i = wp.tid()
    out[i] = a[i] + b[i]
```

wp.tid() identifies which element this invocation owns.

Launching a kernel

```
import numpy as np

device = "cpu"                          # or "cuda:0"
a = wp.array(np.arange(8, dtype=np.float32), device=device)
b = wp.ones(8, dtype=float, device=device)
out = wp.empty_like(a)

wp.launch(add, dim=8, inputs=[a, b], outputs=[out], device=device)
print(out.numpy())                    # [1. 2. 3. 4. 5. 6. 7. 8.]
```

dim=8 creates logical indices 0, 1, ..., 7. Do not call add(a, b, out).

Python scope vs kernel scope

Python scope	Kernel scope
NumPy, files, plotting, argparse	Typed arithmetic and supported Warp built-ins
wp.array(), wp.zeros(), wp.launch()	wp.tid(), wp.dot(), wp.length()
Dynamic Python objects	Arguments with supported, known types
Owns simulation / frame loop	Owns the work for one logical index

Pass simulation data explicitly instead of capturing changing Python globals.

Warp data types

Category	Examples
Signed integers	wp.int32, wp.int64
Unsigned integers	wp.uint32, wp.uint64
Floating point	wp.float16, wp.float32, wp.float64
Vectors / matrices	wp.vec3, wp.mat33
Kernel aliases	int → wp.int32; float → wp.float32

Match kernel annotations to the actual array element type.

Type aliases and casts

```
# In a kernel:
i = wp.tid()                        # integer index
t = float(i) * 0.01                  # explicit int → float cast
j = int(t)                           # truncates toward zero
precise = wp.float64(1.0)

# In Python:
host = np.array([1.0, 2.0], dtype=np.float32)
a = wp.array(host, dtype=wp.float32, device=device)
```

Use 1.0 for float arithmetic; be deliberate when mixing numeric types.

Compound types: vectors

```
# Inside a kernel or @wp.func:
v = wp.vec3(1.0, 2.0, 3.0)
g = wp.vec3(0.0, -9.81, 0.0)
u = v + 0.01 * g

y = u[1]                              # components: 0, 1, 2
speed = wp.length(u)
alignment = wp.dot(u, v)
normal = wp.normalize(v)
```

Position, velocity and force are natural uses of wp.vec3.

Compound types: matrices

```
m = wp.mat33(1.0, 0.0, 0.0,
             0.0, 2.0, 0.0,
             0.0, 0.0, 3.0)
v = wp.vec3(1.0, 2.0, 3.0)

scaled = m * v                    # (1, 4, 9)
entry = m[1, 1]                   # 2.0
mt = wp.transpose(m)
```

Use small matrices for local transformations; arrays hold many such values.

wp.array: allocating data

```
n = 1024
device = "cpu"

mass = wp.ones(n, dtype=float, device=device)
position = wp.zeros(n, dtype=wp.vec3, device=device)
velocity = wp.zeros_like(position)
scratch = wp.empty(n, dtype=float, device=device)

# scratch has no defined values until you write them.
```

Every array has a shape, an element dtype and a memory device.

Array shape vs element shape

Warp allocation	Logical interpretation
wp.zeros(n, dtype=wp.vec3)	n elements; each element is a 3D vector
wp.zeros((n, 3), dtype=float)	n × 3 scalar elements; a 2D array
wp.zeros(n, dtype=wp.mat33)	n elements; each is a 3 × 3 matrix

```
# position: shape (n,), dtype wp.vec3
p = position[i]                    # a vec3
y = p[1]                           # a scalar component
```

Array indexing and write-back

```
@wp.kernel
def lift(position: wp.array(dtype=wp.vec3), dy: float):
    i = wp.tid()
    p = position[i]                 # load one vector value
    p = p + wp.vec3(0.0, dy, 0.0)
    position[i] = p                 # store the updated value

# Python: launch over all vector elements
wp.launch(lift, dim=position.shape[0],
          inputs=[position, 0.1], device=device)
```

Keep every index within the array bounds.

NumPy ↔ Warp

```
host = np.zeros((8, 3), dtype=np.float32)
position = wp.array(host, dtype=wp.vec3, device=device)

print(position.shape)           # (8,), not (8, 3)
snapshot = position.numpy().copy()
print(snapshot.shape)           # (8, 3)

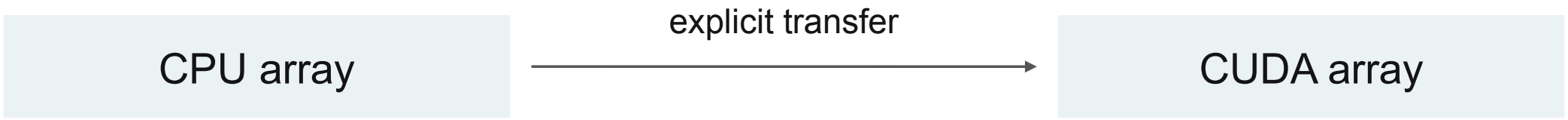
# On CUDA: .numpy() transfers data to the host.
# On CPU: .numpy() may share storage; .copy() saves a snapshot.
```

Convert at the boundaries of the computation, rather than inside every substep.

Arrays live on a device

```
# Requires a compatible CUDA device
host_array = wp.zeros(8, dtype=float, device="cpu")
gpu_array = wp.zeros(8, dtype=float, device="cuda:0")
wp.copy(gpu_array, host_array)

# Kernel arguments must be accessible on the launch device.
# For this tutorial: allocate all participating arrays there.
```



Two-dimensional arrays

```
@wp.kernel
def fill_grid(grid: wp.array2d(dtype=float)):
    i, j = wp.tid()
    grid[i, j] = float(i * 10 + j)

grid = wp.zeros((3, 4), dtype=float, device=device)
wp.launch(fill_grid, dim=grid.shape, inputs=[grid], device=device)
# [[ 0,  1,  2,  3],
#  [10, 11, 12, 13],
#  [20, 21, 22, 23]]
```

The launch dimensionality determines how many indices wp.tid() returns.

Mapping threads to a grid

dim=(3, 4) creates twelve logical work items.

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)

```
i, j = wp.tid()

grid[i, j] = ...
```

For a 1D array of vectors, launch dim=n and use one index.

Matrix size consideration

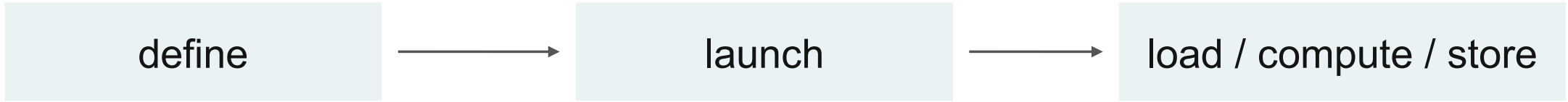
- Small fixed-size matrices are useful per-element values.
- For a large scalar table, keep the large dimensions in the array.
- Matrix value size and array dimensions are different design choices.

```
# Many small transforms:
transforms = wp.zeros(n, dtype=wp.mat33, device=device)

# A large scalar table:
table = wp.zeros((64, 32), dtype=float, device=device)
```

Computation kernel

- @wp.kernel declares a typed computation.
- wp.launch() selects the work size and execution device.
- The first launch compiles code; later launches can reuse cached code.



The compiled subset supports numerical code, rather than arbitrary Python.

Kernel arguments

```
@wp.kernel
def scale(a: wp.array(dtype=float), factor: float,
          out: wp.array(dtype=float)):
    i = wp.tid()
    out[i] = factor * a[i]

wp.launch(scale, dim=a.shape[0], inputs=[a, 2.0],
          outputs=[out], device=device)
```

Arguments may include supported scalars, vectors, matrices, arrays and structs.

Values vs array storage

Argument	Effect of writing inside the kernel
float, int, vec3, mat33	Local value changes do not update Python variables
wp.array(...)	a[i] = ... writes to the array storage
inputs / outputs lists	Describe argument order; do not enforce access rules

```
# factor = 3.0 changes only the local kernel parameter.
# out[i] = 3.0 updates the caller-visible array element.
```

Kernel output: write an array

- Warp kernels do not return a Python value.
- Store the result in an array, then read it from Python.

```
@wp.kernel
def square(a: wp.array(dtype=float), out: wp.array(dtype=float)):
    i = wp.tid()
    out[i] = a[i] * a[i]

wp.launch(square, dim=a.shape[0], inputs=[a, out], device=device)
result = out.numpy()
```

Loops: the launch is the parallel dimension

```
# Conceptually in Python:
for i in range(n):
    out[i] = a[i] + b[i]

# Warp: one invocation owns one i
@wp.kernel
def add(a: wp.array(dtype=float), b: wp.array(dtype=float),
        out: wp.array(dtype=float)):
    i = wp.tid()
    out[i] = a[i] + b[i]
```

Launch with dim=n. A for-loop inside the kernel is not automatically a new grid.

Loops inside a kernel

```
@wp.kernel
def row_sum(a: wp.array2d(dtype=float),
            out: wp.array(dtype=float)):
    i = wp.tid()
    total = float(0.0)
    for j in range(a.shape[1]):
        total = total + a[i, j]
    out[i] = total

# Python: launch dim=a.shape[0] (one work item per row)
```

Rows are independent; each row’s inner loop is sequential work.

Conditionals

```
@wp.kernel
def clamp_nonnegative(a: wp.array(dtype=float)):
    i = wp.tid()
    value = a[i]
    if value < 0.0:
        a[i] = 0.0
    else:
        a[i] = value

# Different logical threads may take different branches.
```

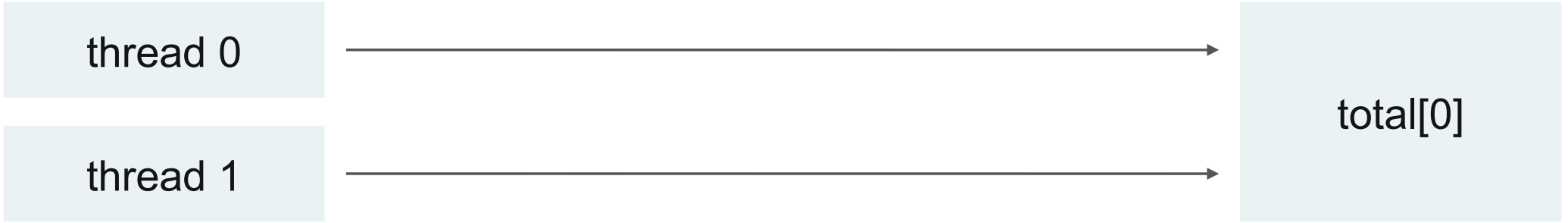
Keep branches correct first; profile before changing them for speed.

Parallel writes: watch for races

Independent output slots are safe; shared updates need coordination.

```
# Good: each i writes a different element
out[i] = a[i] + b[i]

# Unsafe parallel reduction: many i update the same element
total[0] = total[0] + a[i]
```



Atomic operations

```
@wp.kernel
def sum_values(a: wp.array(dtype=float),
               total: wp.array(dtype=float)):
    i = wp.tid()
    wp.atomic_add(total, 0, a[i])

total = wp.zeros(1, dtype=float, device=device)
wp.launch(sum_values, dim=a.shape[0],
          inputs=[a, total], device=device)
```

Atomics prevent lost updates; floating-point sums can still vary in rounding.

Neighbour reads: use two buffers

```
@wp.kernel
def smooth(src: wp.array(dtype=float), dst: wp.array(dtype=float)):
    i = wp.tid()
    if i == 0 or i == src.shape[0] - 1:
        dst[i] = src[i]
    else:
        dst[i] = (src[i - 1] + src[i] + src[i + 1]) / 3.0

# Python: src and dst must refer to separate storage
wp.launch(smooth, dim=src.shape[0], inputs=[src, dst], device=device)
src, dst = dst, src
```

Every thread reads the same old state before the next launch.

Warp functions: @wp.func

```
@wp.func
def kinetic_energy(v: wp.vec3, mass: float) -> float:
    return 0.5 * mass * wp.dot(v, v)

@wp.kernel
def compute_energy(v: wp.array(dtype=wp.vec3), mass: float,
                   energy: wp.array(dtype=float)):
    i = wp.tid()
    energy[i] = kinetic_energy(v[i], mass)
```

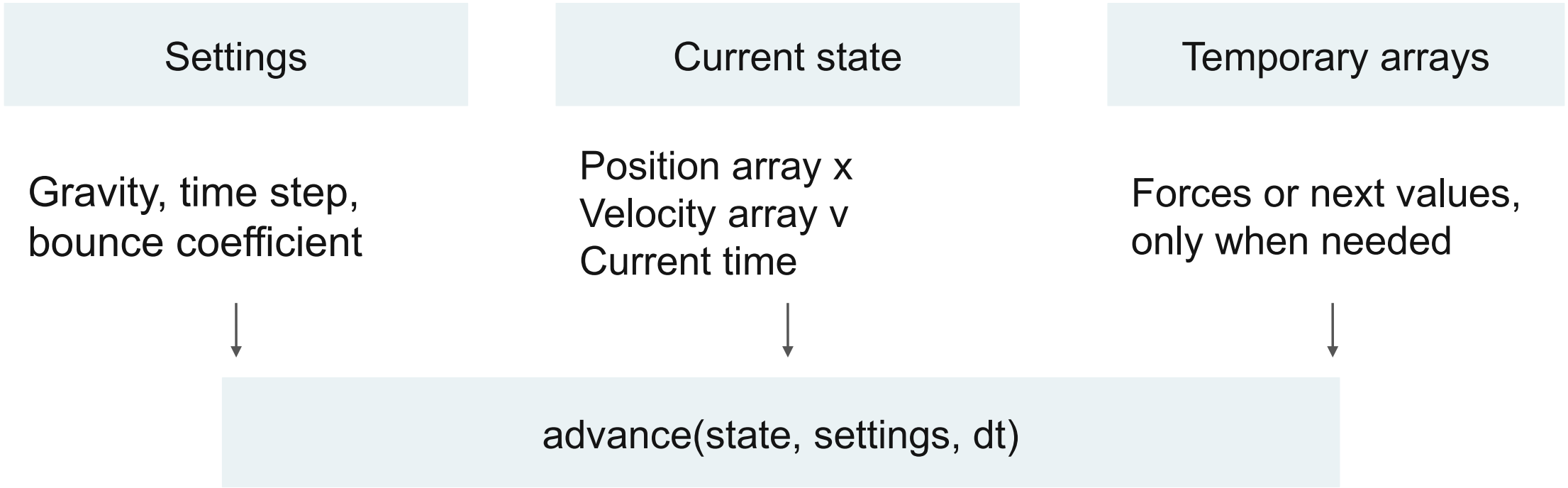
Use @wp.func to share typed numerical logic across kernels.

Kernel vs function

Feature	@wp.kernel	@wp.func
Entry	wp.launch(...)	Call from kernel / function
Thread index	wp.tid()	Pass an index as an argument
Return values	Write output arrays	May return values
Role	Schedule per-element work	Reuse numerical logic

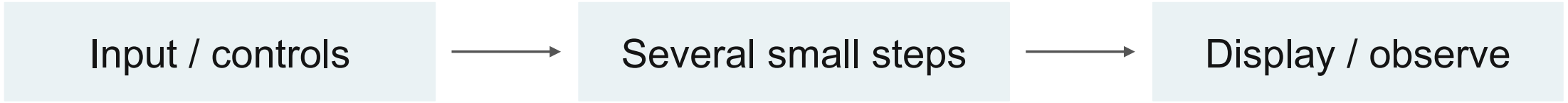
Direct Python calls to @wp.func are experimental; this tutorial calls it in kernels.

Plan the particle simulator



Keep the update separate from displaying or checking the result.

One frame: update, then observe



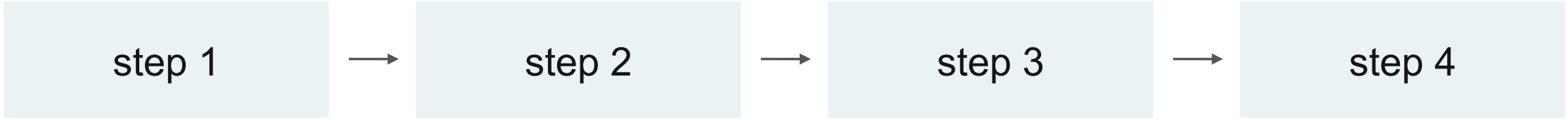
Keep x and v on the device



Reuse the arrays across steps; read back a snapshot when it is time to observe.

Simulation time and display time differ

We will take four small simulation steps before displaying a frame.



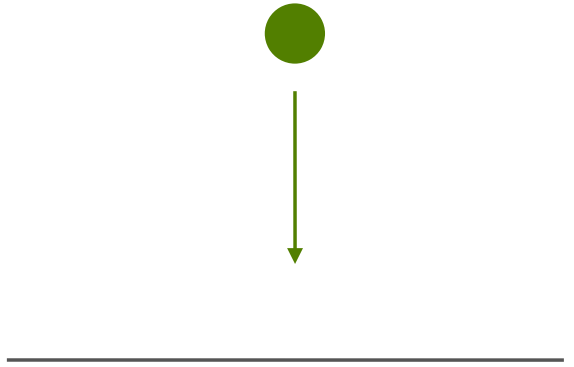
$$4 \times dt = 1/60\ s \quad \rightarrow \quad dt = 1/240\ s$$

- Smaller steps resolve changes in motion more finely.
- Taking more steps per frame costs more computation.

Example: falling particles

- State per particle: position x and velocity v (both vec3).
- Advance velocity, then position: semi-implicit Euler.

```
v_new = v + g · dt
x_new = x + v_new · dt
```



g = (0, −9.81, 0)

If y < 0: project to ground and reverse downward velocity with restitution.

Particle integration kernel

```
@wp.kernel
def integrate(x: wp.array(dtype=wp.vec3), v: wp.array(dtype=wp.vec3),
              dt: float, restitution: float):
    i = wp.tid()
    vi = v[i] + wp.vec3(0.0, -9.81, 0.0) * dt
    xi = x[i] + vi * dt
    if xi[1] < 0.0:
        xi = wp.vec3(xi[0], 0.0, xi[2])
        if vi[1] < 0.0:
            vi = wp.vec3(vi[0], -restitution * vi[1], vi[2])
    x[i] = xi
    v[i] = vi
```

One thread writes only its own particle: no atomic operation is needed.

Host loop: simulation substeps

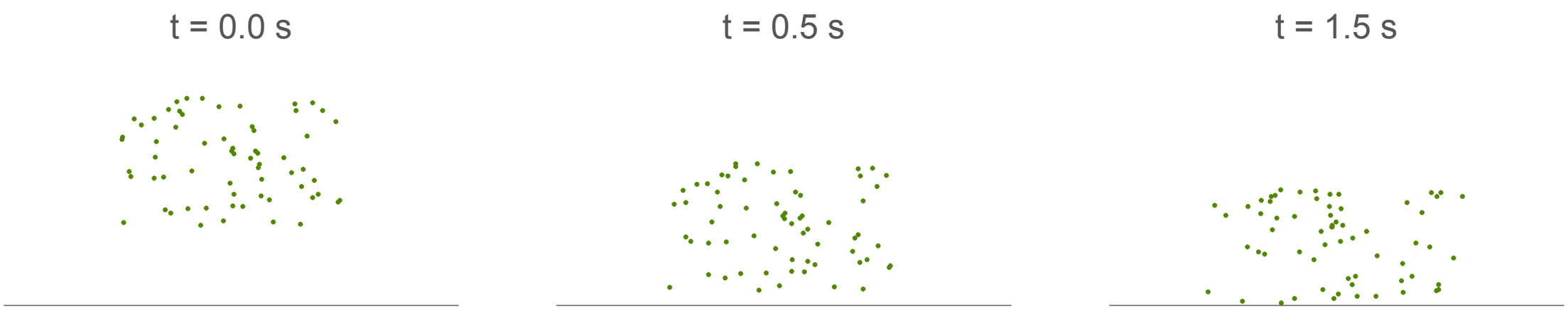
```
n = 64
host_x = np.zeros((n, 3), dtype=np.float32)
host_x[:, 0] = np.linspace(-1.8, 1.8, n)
host_x[:, 1] = np.linspace(1.5, 4.0, n)
x = wp.array(host_x, dtype=wp.vec3, device=device)
v = wp.zeros_like(x)
dt = 1.0 / 240.0

for frame in range(120):
    for substep in range(4):
        wp.launch(integrate, dim=n,
                  inputs=[x, v, dt, 0.75], device=device)
    snapshot = x.numpy().copy()
```

4 × (1/240 s) = 1/60 s of simulated time per displayed frame.

Particle motion and ground contact

- Gravity accelerates particles downward.
- Ground contact reverses downward velocity and reduces rebound height.



y points upward; the floor is at y=0; restitution is 0.75.

Demo: heat diffusion on a grid

Each new temperature depends on the OLD value and four neighbours.




```
u_new = u + alpha · lap
lap = neighbour_sum − 4u

alpha = D · dt / h²
```

For this explicit 2D five-point scheme, use 0 ≤ alpha ≤ 1/4.

Diffusion kernel: read old, write new

```
@wp.kernel
def diffuse(src: wp.array2d(dtype=float),
            dst: wp.array2d(dtype=float), alpha: float):
    i, j = wp.tid()
    n = src.shape[0]
    if i == 0 or j == 0 or i == n-1 or j == n-1:
        dst[i, j] = 0.0
    else:
        lap = src[i-1, j] + src[i+1, j]
        lap = lap + src[i, j-1] + src[i, j+1]
        dst[i, j] = src[i, j] + alpha * (lap - 4.0*src[i, j])
```

This example uses an n × n grid with a fixed zero-temperature boundary.

Diffusion loop: keep two persistent buffers

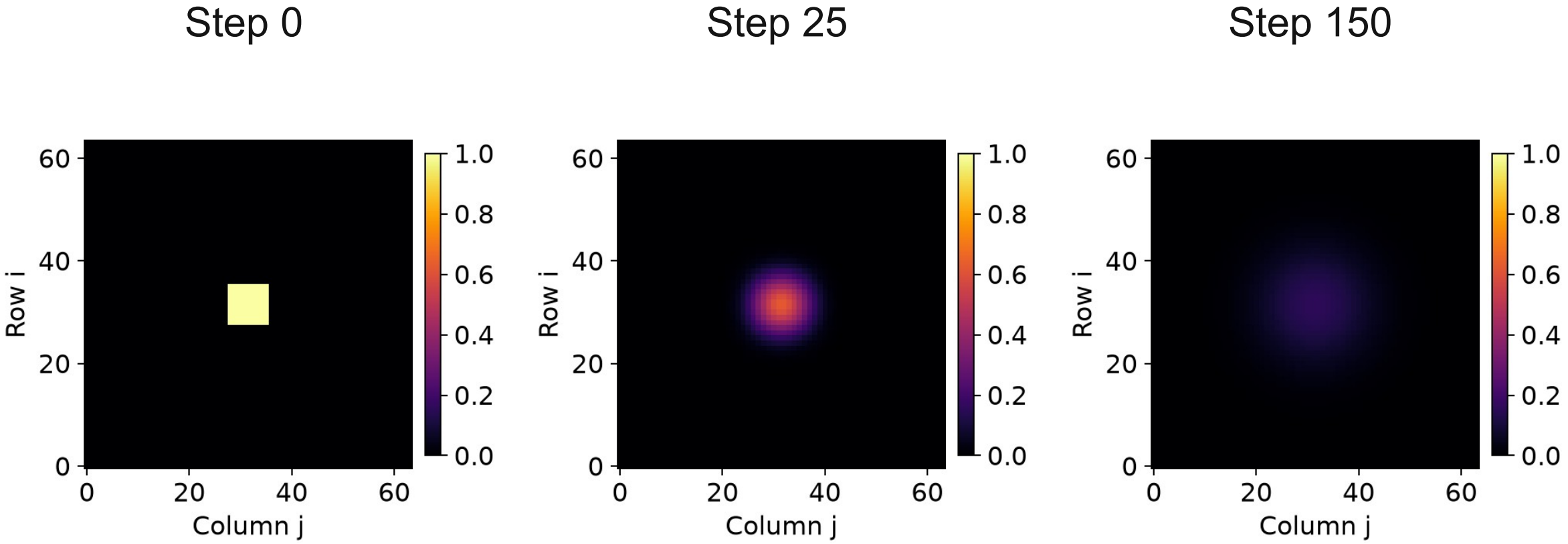
```
host = np.zeros((64, 64), dtype=np.float32)
host[28:36, 28:36] = 1.0
src = wp.array(host, dtype=float, device=device)
dst = wp.empty_like(src)

for step in range(150):
    wp.launch(diffuse, dim=src.shape,
              inputs=[src, dst, 0.2], device=device)
    src, dst = dst, src

temperature = src.numpy().copy()
```

Swapping src and dst changes ownership of old / new state without copying the grid.

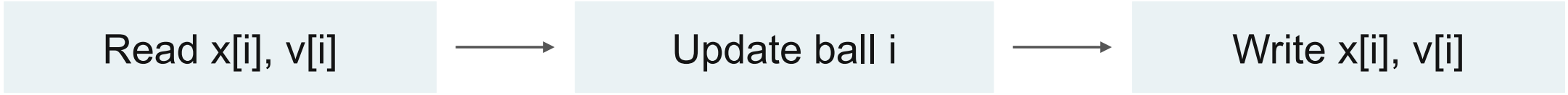
Heat spreads while the peak falls



Same colour scale in all panels. Zero boundary values allow heat to leave the domain.

Why use different buffers in the two demos?

Bouncing particles



No other ball reads this state during the update, so we can overwrite it.

Heat diffusion



The old grid stays unchanged while every new temperature is computed.

Finish the grid, then swap old and new for the next step.

Performance: measure the right work

- Warm up compilation before measuring execution.
- Allocate buffers once; reuse them across steps.
- Measure transfers separately from kernel execution.

```
from time import perf_counter
wp.synchronize_device(device)
t0 = perf_counter()
# Launch already-compiled kernels here.
wp.synchronize_device(device)
elapsed = perf_counter() - t0
```

Debugging checklist

Symptom	Check
Type mismatch	np.float32 vs np.float64; scalar / vector dtype
Wrong thread indices	dim matches array shape and wp.tid() arity
Out-of-bounds access	Index limits and launch dimensions
Unstable results	Shared writes, neighbour reads, uninitialized memory
CUDA device unavailable	Driver version and wheel CUDA variant

Start small; compare to NumPy; then verify the CUDA path.

Differentiable kernels

- Record a scalar objective and compute its gradient.

```
@wp.kernel
def squared_loss(x: wp.array(dtype=float),
                 loss: wp.array(dtype=float)):
    i = wp.tid()
    wp.atomic_add(loss, 0, x[i] * x[i])

# L = sum_i x[i]^2
# dL/dx[i] = 2*x[i]
```

Use a separate smooth example; contact branches need additional care.

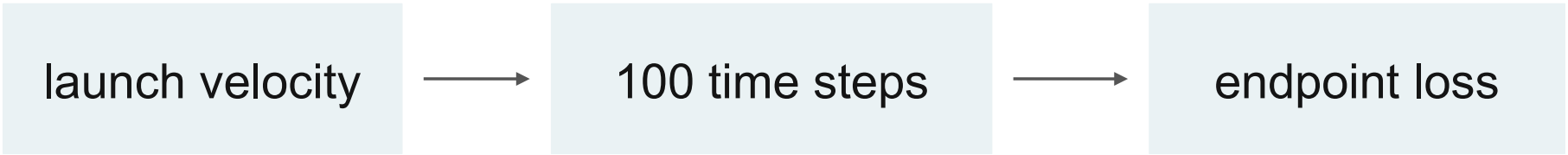
wp.Tape

```
x = wp.array([1.0, 2.0, 3.0], dtype=float, device=device,
             requires_grad=True)
loss = wp.zeros(1, dtype=float, device=device, requires_grad=True)
tape = wp.Tape()
with tape:
    wp.launch(squared_loss, dim=3, inputs=[x, loss], device=device)
tape.backward(loss=loss)
print(loss.numpy())             # [14.]
print(x.grad.numpy())           # [2. 4. 6.]
```

Preserve forward values needed by backward; zero loss and gradients before reuse.

Demo: find a launch velocity from a target

- Given: initial height 0, gravity −9.81, target height 1 m after 1 s.
- Unknown: initial upward velocity. Minimise the squared endpoint error.



gradient → update velocity → simulate again

A smooth flight model isolates differentiation through time; there is no ground contact.

A differentiable step and endpoint loss

```
@wp.kernel
def flight(y0: wp.array(dtype=float), v0: wp.array(dtype=float),
           y1: wp.array(dtype=float), v1: wp.array(dtype=float),
           dt: float):
    i = wp.tid()
    v1[i] = v0[i] - 9.81 * dt
    y1[i] = y0[i] + v1[i] * dt

@wp.kernel
def endpoint_loss(y: wp.array(dtype=float), target: float,
                  loss: wp.array(dtype=float)):
    d = y[0] - target
    loss[0] = d * d
```

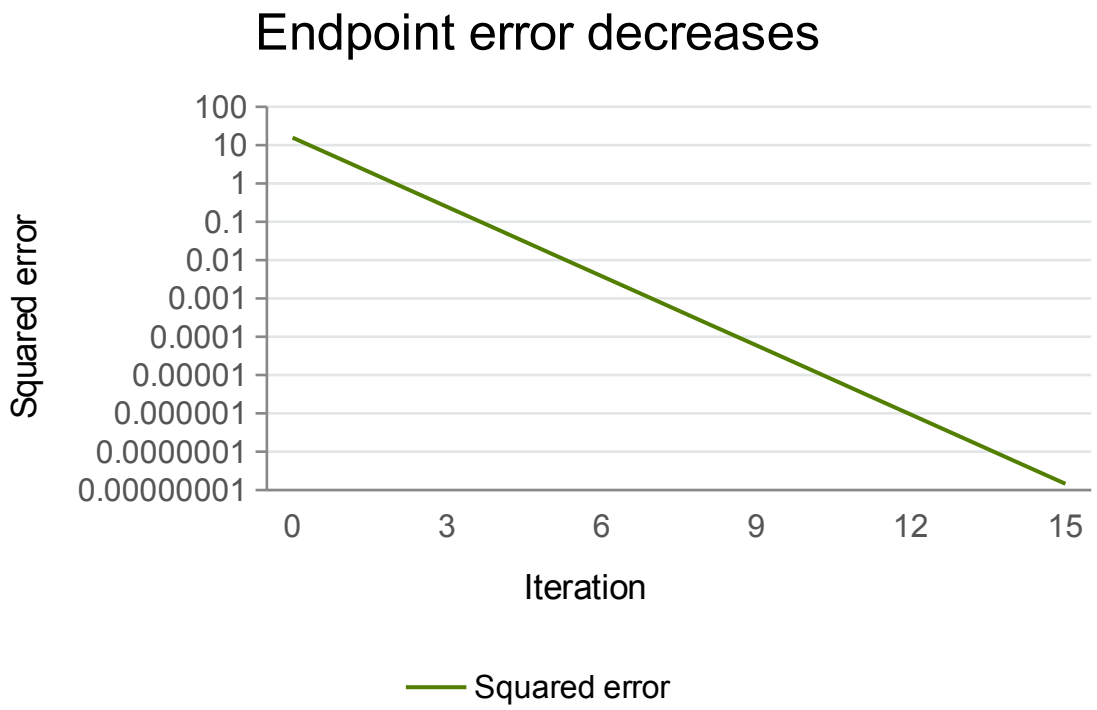
The numerical rollout is part of the objective being differentiated.

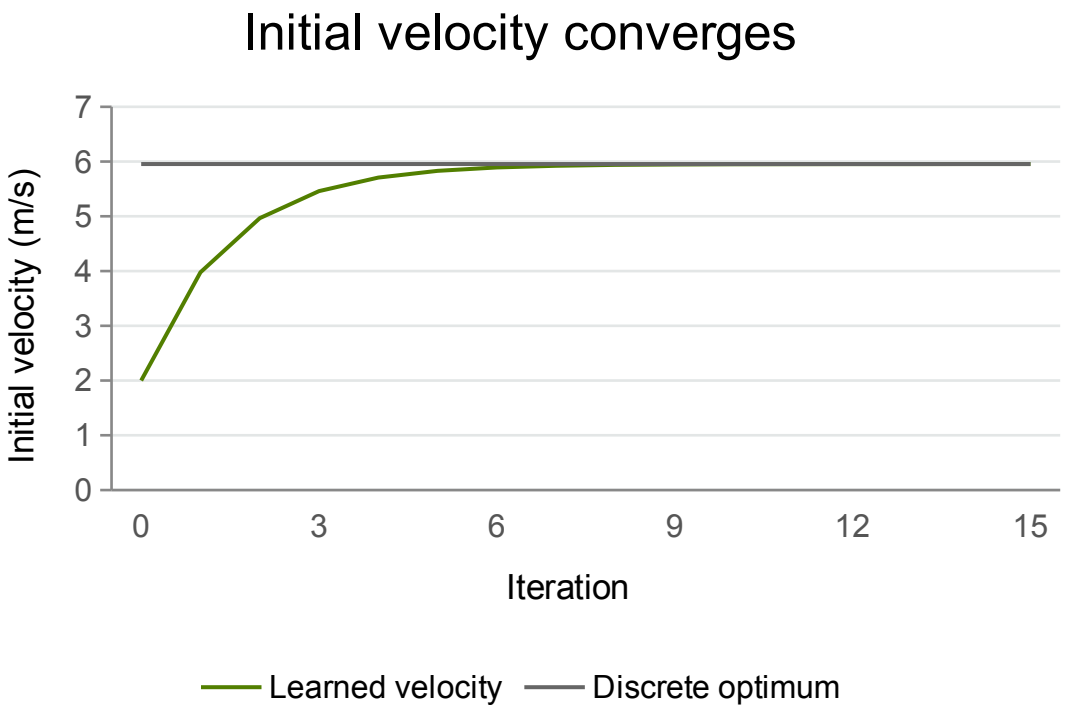
Keep the forward trajectory for backward

```
y = [wp.zeros(1, device=device, requires_grad=True)
     for t in range(101)]
v = [wp.zeros(1, device=device, requires_grad=True)
     for t in range(101)]
v[0].assign(np.array([2.0], dtype=np.float32))
loss = wp.zeros(1, device=device, requires_grad=True)
with wp.Tape() as tape:
    for t in range(100):
        wp.launch(flight, dim=1, device=device,
                  inputs=[y[t], v[t], y[t+1], v[t+1], 0.01])
    wp.launch(endpoint_loss, dim=1,
              inputs=[y[-1], 1.0, loss], device=device)
tape.backward(loss=loss)
```

Separate time states preserve forward values needed by the reverse pass.

Optimisation learns a useful initial velocity






```
v0 ← v0 − 0.25 · gradient
```

Zero gradients between iterations. The discrete optimum is approximately 5.954 m/s.

The same update: array operations or a kernel?

NumPy: express the work over whole arrays.

```
v = v + np.array([0.0, -9.81, 0.0], dtype=np.float32) * dt
x = x + v * dt
```

Warp: express the work for one logical item.

```
i = wp.tid()
vi = v[i] + wp.vec3(0.0, -9.81, 0.0) * dt
x[i] = x[i] + vi * dt
v[i] = vi
```

Same model and integrator; different abstractions for exposing parallel work.

CPU options: Python loops, NumPy and Numba

Approach	Useful when	What to watch
Python loop	Tiny prototypes; readable baseline	Per-element interpreter overhead
NumPy arrays	Dense array operations; CPU workflows	Temporary arrays and memory traffic
Numba CPU JIT	Custom loops over NumPy data	Compilation; dependency-safe prange
Warp CPU kernels	Validate the same kernel logic locally	CPU execution ≠ CUDA throughput

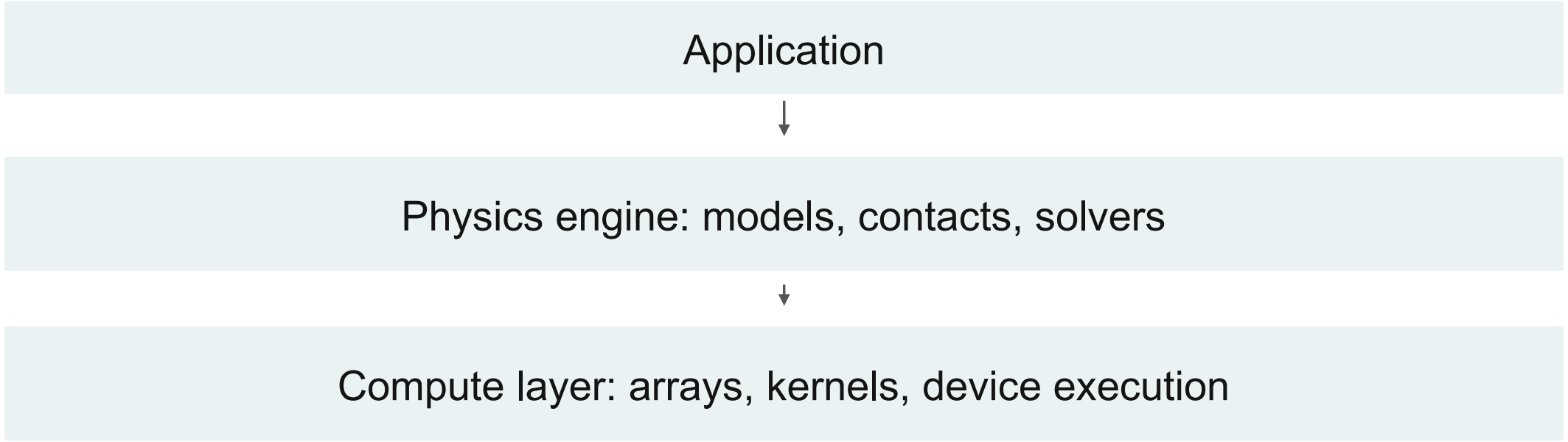
Start with the simplest correct implementation that matches the workload.

GPU options: tensors, Warp and CUDA C++

Approach	Natural unit of work	Main trade-off
PyTorch + compile	Tensor operations; autodiff graphs	Strong ML integration; compiler constraints
Warp	Typed per-item / spatial kernels	Explicit dataflow; supported kernel subset
CUDA C++	Threads, blocks and memory hierarchy	Fine control; more implementation effort

Tensor compilers can fuse operations too. Compare measured workloads, not language labels.

A compute framework vs a physics engine



Use an engine for established mechanics; use a compute framework to build custom methods.

For example, Newton is a physics engine built on Warp. These layers can work together.

Choose tools by the bottleneck

If the work is mostly...	A reasonable direction
Small CPU arrays or dense linear algebra	NumPy and optimised numerical libraries
Custom CPU loops	Numba CPU JIT or C++
Tensor-heavy ML and differentiable models	PyTorch; consider compilation
Custom spatial / per-element GPU work	Warp; CUDA C++ if more control is needed
A standard physical system	A physics engine with a suitable solver

Benchmark equal accuracy, size and precision; include transfers and startup when relevant.